بسم الله الرحمن الرحیم

فاز صفر پروژه شی گرایی 1404

نام و نام خانوادگی: سید کیارش اشراقی

شماره دانشجویی: 403101383

- 1.1.5 پرسش 1:

نخست کلاس Node را برای محل اتصالات عناصر مدار لحاظ می‌کنیم که دارای داده‌های پرایوت id و وvoltage هستند. البته که با توجه به اینکه ولتاژ می‌تواند متغیر با زمان باشد و ما برای محاسبات نیاز به ولتاژهای زمان های قبل داریم، از یک مپ برای پیر کردن ولتاژ و زمان استفاده می‌کنیم.
علاوه بر متد کانستراکتور، متدهای getter آیدی و voltage را داریم، که برای voltage، ولتاژ آن زمان داده شده را برمی‌گردانیم.
همچنین به setter ولتاژ نیاز داریم که با گرفتن ولتاژ و زمان آن، آنها را در مپ پیر می‌کند و قرار می‌دهد.

همچنین کلاس Element را داریم که برای استفاده از عناصر در مدار قرار داده شده است. این کلاس دارای 2 تا اشاره گر به کلاس Node به صورت پروتکتد هستند که شامل دو سر عنصر می‌شوند. همینطور به توجه به مفهوم چندریختی، یکسری متد مجازی همراه با virtual اش را برایش تعریف کرده ایم که بین تمام کلاس های مشتق شده از این کلاس، یکسان است. متدهایی که فعلا برای تحلیل های ما لازم است یکی بدست آوردن جریان گذرا از عناصر با توجه به ولتاژ گره هایشان است و مابقی هم که getter هستند.

در ادامه کلاس resistor را داریم که یک نوعی از عناصر است که دارای داده resistance مقاومت است، همراه با داده های خود کلاس Element. همینطور یک متد getValue برای برگرداند مقدار resistance و یک متد getType هم برای نشان دادن نوع عنصر داریم. در آخر هم متد حساب جریان را برای Element که درین جا هست.

حال می‌رسیم به یک نوع عنصر دیگر یعنی voltageSource که دارای یک داده voltage ثابت است.
همین طور متد کانستراکتور را برای این کلاس داریم. برای متد اصلی ما یعنی حساب جریان شاخه، با توجه به دانسته‌های مداری خود، می‌دانیم که جریان گذرنده از این شاخه، کاملا بستگی به مداری است که این عنصر در آن قرار دارد. لذا یکسری محاسبات عددی برای حل مدار طبق معادلات باید انجام شود تا جریان این شاخه بدست بیاید.
همانند کلاس resistance، این کلاس هم همان متد های خاصش را دارد.

برای کلاس CurrentSource هم همین منوال کلاس بالا را داریم که منتها برای متد حساب جریان، به راحتی خود مقدار داده کلاس یعنی current را بر می‌گردانیم.

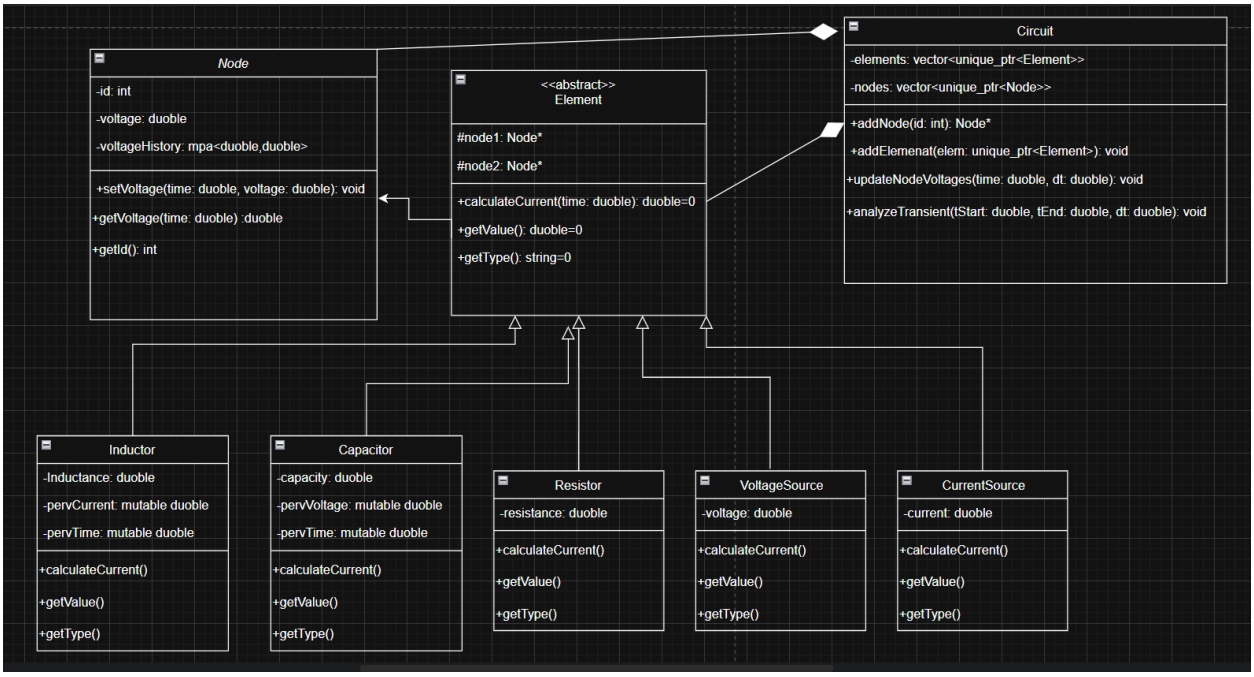
حال برای کلاس Capacitor هم، مقدار داده capacity را داریم همراه با دوتا متغیر برای نگه داری ولتاژ و زمان پیشین که در کانستراکتور صفر در نظر گرفته شده اند. برای متد اصلی حساب جریان، نیاز به یک الگوریتم داریم که با حساب اختلاف ولتاژ و زمان، جریان جدید را حساب می کنیم و زمان و ولتاژ حال حاضر را در ولتاژ و زمان قبلی نگه می داریم.

در آخر هم می‌رسیم به کلاس Inductor که شامل مقدار Inductance و همان مقادیر نگهداری زمان و جریان است. با توجه به منطق مداری اش، نیازی به عملیات انتگرال برای حساب جریان عبوری اش دارد. که درین جا هم با استفاده از تقریب خطی و کمی شبیه همین متد برای کلاس Capacitor و به طور لحظه به لحظه، تغییرات کوچک مدار را حساب می‌کنیم که در کلاس Circuit در لحظه تعادل بدست می‌آید. مابقی متدها هم مثل عناصر دیگر است.

در آخر هم می‌رسیم به کلاس Circuit، که قرار است در آن با متدهای مخصوصش، فعلا بتوانیم گره اضافه کنیم و در نهایت، مدار را آنالیز کنیم. همچنین این کلاس شامل دو وکتور از اشاره گرها به کلاس‌های Node و Element است.
البته از یک متدی هم برای تغییر وضعیت مدار با توجه به زمان حال حاضر و میزان تغییراتمان استفاده کرده‌ایم که باز با روش های تقریبی و عددی کار می‌کند.
برای آنالیز مدار هم، با همان روش عددی، در هر لحظه، تغییرات را بدست می‌آوریم و با اعمال روی عناصر، در نهایت نتایج را بدست می‌آوریم.

- 2.1.5 پرسش 2:

نمودار UML در سایت ذکر شده در گروه، زده شده است و بدین گونه است:



- ### 1.2.5 پرسش 3:

بخش خوبی از توضیحات مربوط به کارکرد هر متد در کلاس‌ها، در پرسش اول پاسخ داده شد، به هر حال بنده درین جا به طور مختصر، توضیحات خاص تر مربوط به هر کلاس را ارائه می‌کنم:

کلاس Node: متد گتر ولتاژ این کلاس، با گرفتن زمان مد نظر، ولتاژ آن زمان که در آن گره وجود دارد را (موجود در مپ کلاس) بر می‌گرداند.
همچنین برای داشتن ساختار انعطاف پذیر، محاسبات ولتاژ خارج از این کلاس است، لذا باید یک ستر برای ولتاژ هم داشته باشیم که یک عضو جدید با گرفتن زمان و ولتاژ مورد نظر به مپ پیر کننده ولتاژ و زمان اضافه می‌کند.

کلاس Element: همان طور که در سوال گفته شده است، برای این کلاس، یک کلاس ابسترک طراحی کرده‌ایم که یکسری متد یکسان برای همه عناصر در نظر گرفته است. متد حساب جریان که برای تحلیل الزامی است، و متدهای گت ولیو و گت تایپ که گتر هستند.

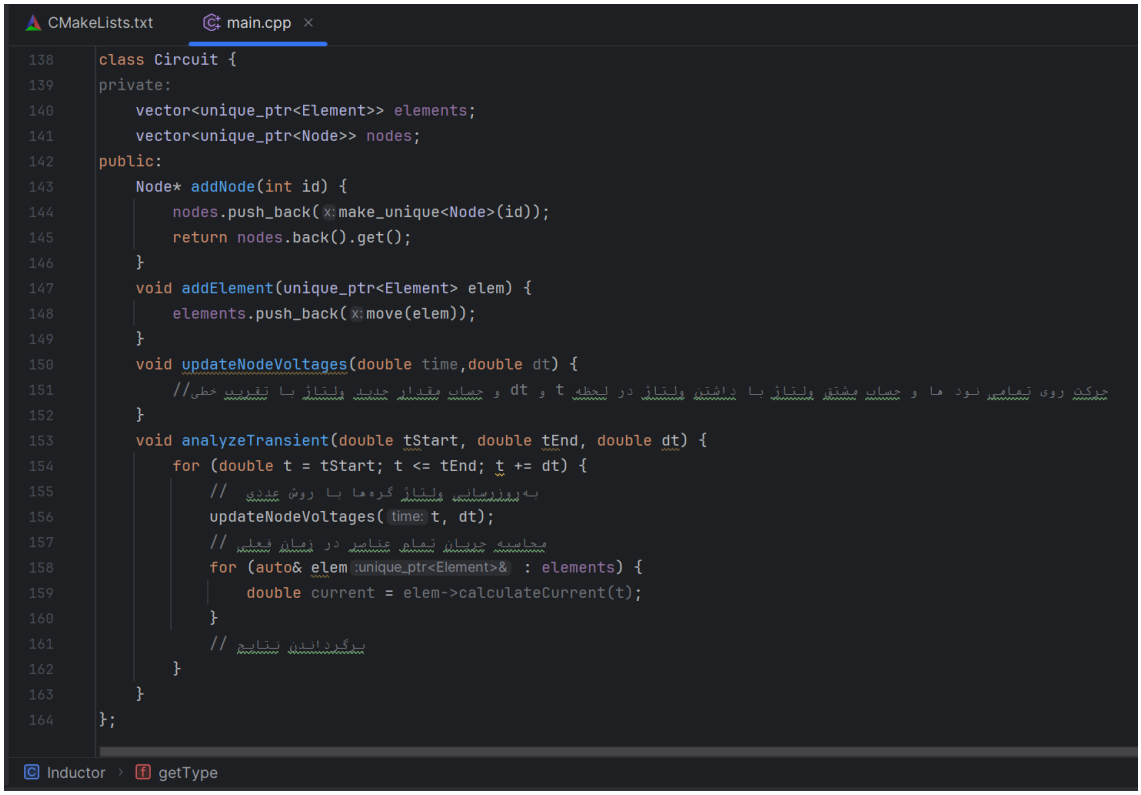
کلاس‌های مشتق شده از Element: متدهای گت ولیو و گت تایپ، مقادیر خاص هر کدام را بر می‌گرداند مثل مقدار resistance مقاومت. اما برای حساب جریان، برای Rsistor، با قانون اهم جریان را با توجه به

ولتاژ شاخه‌ها بر می‌گرداند. برای کلاس‌های Capacitor و Inductor هم دیدیم که با یکسری محاسبات تقریبی، در یک بازه زمانی کوچک، با منطق مداری، جریان را حساب می‌کنیم.
برای voltageSource هم که می‌خواهیم جریانش را حساب کنیم، باید با حل معادلات مداری، این مقدار تعیین شود که درین صورت نیاز به است به محاسبات عددی.

کلاس Circuit: درین کلاس، یک متد اد نود داریم که برای ساختن گره های مدار نیاز است. همین طور متد اد المنت که برای تعیین نوع قطعه بین گره ها لازم است. همین طور برای انجام محاسبات عددی، یک کلاس آپدیت نود ولتاژ داریم که در هر لحظه با توجه به متد حساب جریان کلاس ها و قوانین KCL و KVL، ولتاژها را بدست می‌آورد. در آخر هم آنالیز را داریم که با متد آپدیت نود ولتاژ، روی کل عناصر حرکت می‌کند و آنها را تحلیل می‌کند و نتایج نهایی را بر می‌گرداند.

- 2.2.5 پرسش 4:

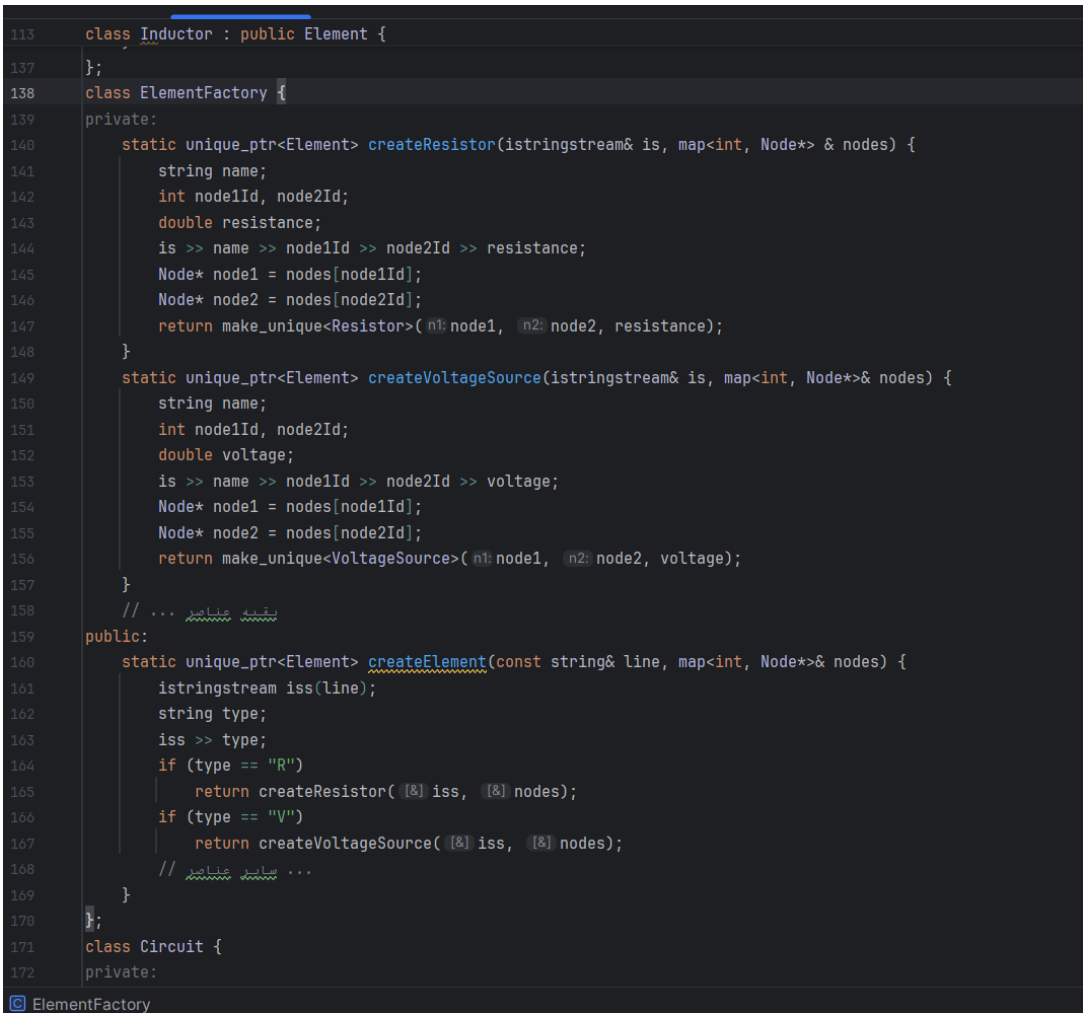
چون کلاس‌های مشتق شده از کلاس Element، دارای یکسری متد یکسان و رفتار یکسان هستند، از مفهوم چندریختی استفاده می‌کنیم. یعنی اینطوری که اشاره گرهایی از المنت ها را نگه می‌داریم در یک وکتوری و بعدا مثلا در متد آنالیز، با هر پوینتر یکسان رفتار می‌شود و به راحتی از متد حساب جریانش می‌توان استفاده کرد. که این سبب می‌شود که بتوانیم بدون پیاده سازی انواع کلاس‌های مختلف و متدهای مختلف و حالت بندی متدهای کلاس Circuit، الگوریتممان را پیاده کنیم.
درین پایین هم نمونه کدی که از این مفهوم استفاده کرده است آورده شده است:



- 1.3.5 پرسش 5:

به نظر می رسد برای این سطح از برنامه، استفاده از الگوی Factory Method مناسب تر است چرا که هم به نسبت موارد پیشنهاد شده در داک، از سطح پیچیدگی پیاده سازی کمتر برخوردار است و هم اینکه هرکدام از عناصر از هم مستقل هستند و با این الگو می شود اشیا را به طور جدا ساخت و کنترل کرد.

درین زیر نمونه استفاده از این الگو را برای برنامه خود آورده‌ایم به طوری که دستورات را به صورت رشته می‌گیریم و داده‌های لازمه برای هر عنصر یا گره را جدا می‌کنیم و آبجکت مربوطه را می‌سازیم.



```
class Inductor : public Element {
};
class ElementFactory {
private:
    static unique_ptr<Element> createResistor(istringstream& is, map<int, Node*> & nodes) {
        string name;
        int node1Id, node2Id;
        double resistance;
        is >> name >> node1Id >> node2Id >> resistance;
        Node* node1 = nodes[node1Id];
        Node* node2 = nodes[node2Id];
        return make_unique<Resistor>( n1: node1,  n2: node2, resistance);
    }
    static unique_ptr<Element> createVoltageSource(istringstream& is, map<int, Node*>& nodes) {
        string name;
        int node1Id, node2Id;
        double voltage;
        is >> name >> node1Id >> node2Id >> voltage;
        Node* node1 = nodes[node1Id];
        Node* node2 = nodes[node2Id];
        return make_unique<VoltageSource>( n1: node1,  n2: node2, voltage);
    }
    // ... بقیه عناصر
public:
    static unique_ptr<Element> createElement(const string& line, map<int, Node*>& nodes) {
        istringstream iss(line);
        string type;
        iss >> type;
        if (type == "R")
            return createResistor( [&] iss,  [&] nodes);
        if (type == "V")
            return createVoltageSource( [&] iss,  [&] nodes);
        // سایر عناصر ...
    }
};
class Circuit {
private:
```

ElementFactory

```
    void updateNodeVoltages(double time,double dt) {
        //خطی با تقریب ولتاژ جدید مقدار حساب و dt و t لحظه در ولتاژ با داشتن مشتق ولتاژ حساب ها نود تمامی روی حرکت
    }
    void getFromSPICE(const vector<string>& lines) {
        map<int, Node*> nodes;
        for (const auto& line :const string&  : lines) {
            auto element :unique_ptr<Element>  = ElementFactory::createElement(line,  [&] nodes);
            elements.push_back( x: move(element));
        }
    }
    void analyzeTransient(double tStart, double tEnd, double dt) {
```

- ### 2.3.5 پرسش 6:

معادلات مدار با استفاده از قوانین KVL و KCL و البته معادله مشخصه عنصر بدست می‌آیند که روش‌های معمول جبرخطی، برای حل معادلات جبری (بدون وجود مشتقات متغیرها) استفاده می‌شوند. یکی از این روش‌ها که در جبرخطی کاربرد زیادی دارد، قاعده کرامر است که البته نیاز به پیاده سازی الگوریتم حساب دترمینان یک ماتریس است که می‌تواند، به شدت طولانی شود مخصوصا برای تعداد معادلات بالا.

اما الگوریتم‌های پایه‌ای و اصلی برای حل این معادلات، همان دو روش گفته شده در صورت سوال داک است:

1. روش حذف گاوسی: درین روش ماتریس ضرایب تشکیل داده می‌شود، سپس با عملیات‌های جمع و تفریق ستون‌ها و سطرها با ضریبی از ستون یا سطر دیگر، این ماتریس را به یک ماتریس بالا مثلثی تبدیل می کنیم، درین صورت به ترتیب بدست آوردن متغیرها از اول به مراتب ساده تر می‌شود (معادله اول، اولین مجهول را می‌دهد، لذا معادله دوم با این حساب دومین مجهول و الی آخر)
مزیت این روش برای ماتریس‌های به نسبت کوچک است، چرا که اردر زمانی این الگورتیم O(n^3) است.
2. روش LU: درین روش، ماتریس ضرایب را به صورت ضرب دو تا ماتریس بالامثلثی و پایین مثلثی در می آوریم یا به اصطلاح تجزیه می‌کنیم. سپس مانند روش قبل با ضرب به ترتیب ماتریس‌ها در بردار متغیرهای ما، معادلات را برای متغیر ها به ترتیب مستقل می‌کنیم. (یعنی باز معادله اول شامل متغیر اول، معادله دوم شامل متغیر دوم و اول و الی آخر).
درین روش یک جورایی از dp استفاده کرده به گونه ای که اردر حافظه را افزایش داده ایم و اردر زمانی را به O(n^2) کاهش داده ایم.

در کل اگر بخواهیم حالات گذرا مدارهایی که دارای مشتقات متغیرهاست، با این روش‌ها حل کنیم، می‌بایست در هر لحظه و با استفاده از تقریب‌های خطی، این ماتریس ها را تشکیل داد و حل کرد. که البته به نظر طولانی می‌رسد.

- 3.3.5 پرسش 7:

با توجه به اینکه تعداد معادلات می تواند بعضا به حدود 100 یا حتی 1000 تا برسد، استفاده از الگوریتم های دستی بالا، می تواند علاوه بر طولانی بودن پیاده سازی اش و زمان اجرایش، دارای مشکلاتی در پیاده سازی باشد که درین صورت خطای شدیدی ایجاد می کند. برای همین می تواند از الگوریتم کتابخانه های سی پلاس پلاس استفاده کرد، مثل کتابخانه Eigen یا Armadillo یا Intel MKL استفاده کرد که دارای الگوریتم های بهینه برای حل این دستگاه معادلات دارند. اما از طرفی استفاده از این کتابخانه ها انعطاف پذیری کد را کاهش می دهد.
اما با استفاده از الگوی Bridge می توان یک کلاس اصلی برای حل تعریف کرد، سپس برای هر کدام از روش های حل چه کتابخانه ها و چه روش های خصوصی، یک کلاسی ارث گرفته می سازیم و الگوریتمشان را پیاده سازی می کنیم. در نهایت می توان با توجه به ماتریس ضرایب مسئله و شرایط دیگر، در متد آنالیز کلاس Circuit از آنها استفاده کرد.

- 4.3.5 پرسش 8:

با توجه به اینکه نمی توان به طور مستقیم معادلات دیفرانسیل را حل کرد، نیاز به حل عددی داریم که هر کدام مزایا و معایب خودشان را دارند:

1. روش اویلر: درین روش که روش ساده و سریع البته کم دقتی است، با استفاده از کوچک کردن گام های حرکت روی زمان برای حساب مشتق یا انتگرال، از تقریب خطی استفاده می کند و اینجوری نقطه به نقطه حرکت می کند و و در نهایت به بازه ای می رسد که مدار پایدار است.
2. روش رونگه_کوتا: درین روش با دقت بالاتر و البته الگوریتم پیچیده تر و طولانی تر، چند نقطه نزدیک بررسی می شوند و با استفاده از میانگین گیری وزن دار، مشتق را بسیار ساده تر تقریب می زند. البته درین روش هم باز به نحوی تقریب خطی است.

در کل روش رونگه-کوتا، برای حل با دقت بالا و البته معادلات پیچیده، مفید تر است.

- 5.3.5 پرسش 9:

در سطح کاری ما، استفاده از الگوی طراحی MVC می‌تواند مفید باشد از این جهت که ساختار برنامه انعطاف پذیر و اجزا مستقل از هم می‌شوند. همچنین برای نمایش نمودارها، برای اینکه ظاهر قشنگی داشته باشد و البته سریع تر باشد یا حتی اینکه بتواند برای تحلیل‌های مختلف، نمودارهای متفاوتی استفاده کرد، کتابخانه‌های ImGui و Qt استفاده می‌کنیم که با این الگوی طراحی هم سازگاری خوبی دارند.
درین الگو، کلاس‌های کنترلر به طور مستقل با استفاده از کلاس های Modal، تحلیل ها را انجام می‌دهند و نتایج تحلیل را به کلاس ویو می‌دهد که آن هم با گرافیک، نمودار را می‌کشد. همان طور که دیده می‌شود انعطاف این طراحی بالاست و می توان به انواع مختلف نمودارها را نمایش داد، که برای تحلیل مدار در حالت گذرا و دائمی پرکاربرد است.

- 5.4.5 پرسش 10:

همان طور که در صورت سوال گفته شد، یک کلاس SPICE-Exception قرار می‌دهیم و یکسری کلاس که انواع خطاها را شامل می‌شوند مثل کلاس‌های
ParsingException,CircuitException,AnalysisException که مثلا برای خطاهای پارس کردن، با توجه به الگوی دریافت دستورات، می‌تواند اینها باشند که مقادیر مقاومت و غیره عدد نباشند یا تعداد ورودی‌ها درست نباشد، که درین صورت باید در همان متد دریافت دستورات در کلاس Circuit، این موارد بررسی شوند و متن خطای مربوطه را با کلاس پارس اکسپشن، پرتاب کند به سمت کلاس ویو. برای خطاهای طراحی هم می توان به ساخت مثلا یک گره نامعتبر یا دادن مقادیر نامتعبر ولتاژ اشاره کرد. برای خطاهای آنالیز مدار، می توان به گام‌های زمانی اشتباه، تقسیم بر صفر کردن در گام‌های زمانی انتهایی مثلا یا غیره اشاره

کرد. برای دریافت این خطاها هم در کلاس ویو، یک ترای می‌گذاریم که در اسکوپ آن، همه دستورات اجرا می‌شوند. در کچ، خود کلاس SPICE-Exception را با آدرس می‌گیریم و خطای مورد نظر پرتاب شده را نمایش می‌دهیم.

- 5.3.5 پرسش 11:

درین جا ما از الگوی کامند برای اجرای برنامه استفاده می‌کنیم. که یک لایه رابطه کاربری دارد همراه دارد با لایه کنترلری که ارتباط با لایه CLI و مدل دارد و دستورات را مدیریت می‌کند. منطق پیاده سازی این الگو، بدین گونه است که یک کلاس داریم که درآن دستورات به طور کلی تعریف می‌شوند. سپس با توجه به مفهوم هم ریختی، با دستورات مختلف مثل اجرا یا استاپ و غیره به یک شکل برخورد می‌شود و در کلاس کنترلر بررسی می‌شوند. که سازگار با طراحی MVC است که در موارد دیگر هم ازش استفاده کرده بودیم.
در کل می‌شود گفت که این ساختار لایه بندی رابط کاربری، روند اجرا برنامه را بسیار منعطف می‌کند و علاوه بر اینکه کد با دسترسی عجیب و غریب کلاس ها به هم، پیچیده نمی‌شود، کلی دستور دیگر را می توان پیاده کرد.